

INTERVIEW PREPARATION HANDBOOK

JusBill

GST Invoicing & Business Intelligence Platform for Indian Retailers

8

SECTIONS

25+

INTERVIEW Q&A

14K+

LINES OF CODE

~45

PAGES

React + Vite

Node.js + Express

MongoDB + Mongoose

Redis (Upstash)

Puppeteer

Groq AI (Llama 3.1)

Docker + Nginx

Cloudinary

Mailjet

Google Cloud VM

CONFIDENTIAL · FOR INTERVIEW PREPARATION ONLY

Table of Contents

Complete Interview Preparation Guide — JusBill Project

1	Project Overview Problem statement, target users, features, tech stack, end-to-end flow	Pages 3–5
2	Project Architecture High-level architecture, layered design, auth flow, deployment architecture	Pages 6–11
3	System Design Module interactions, design tradeoffs, performance, security, future scope	Pages 12–17
4	Database Design All 7 schemas, ER diagram, indexes, relationships, DFD Level 0 & 1	Pages 18–24
5	Critical Code Walkthrough 12 most important files with line-by-line explanation of critical snippets	Pages 25–32
6	Complete Feature Flows Registration, Login, Invoice creation, AI chat, PDF generation, email, upload	Pages 33–38
7	Project-Specific Interview Q&A 25 high-quality questions directly tied to actual code in this repository	Pages 39–47
8	Final Revision Sheet APIs, models, middleware, key decisions — revise 30 min before interview	Pages 48–50



Project Overview

Problem statement · Target users · Features · Tech stack · End-to-end flow

1.1 Problem Statement

Small Indian retailers, kiranas, and shop owners face a significant operational challenge: managing GST-compliant billing, real-time inventory, customer records, and financial analytics manually through paper ledgers or disconnected spreadsheets. Existing solutions are either too expensive (Tally, Zoho Books) or too complex for a single-owner retail shop.

JusBill solves this by providing a zero-learning-curve, browser-based SaaS platform that allows shop owners to create GST invoices, track stock in real-time, manage customers, record purchases, visualize analytics, and interact with an AI business assistant — all from a single dashboard.

1.2 Target Users

PRIMARY

- Kirana / general store owners
- Electronics retailers
- Clothing & textile shops
- Pharmaceutical retail

SECONDARY

- Wholesalers & distributors
- Service-based businesses
- Small manufacturing units

KEY PERSONA

- Non-tech-savvy owner
- GST-registered business
- Monthly revenue: ₹50K–₹10L

1.3 Core Features

Feature	Description	Key Technology
GST Invoice Generation	Create, download, and email GST-compliant invoices with auto-calculated subtotal, discount, GST, and grand total	React, Puppeteer, Mailjet
Inventory Management	Real-time stock tracking. Stock auto-decrements on invoice creation, increments on purchase entry. Low-stock alerts	MongoDB \$inc, Redis Cache
Customer Management	Customer database with GSTIN validation, full-text search, and invoice history	MongoDB text index
Purchase Tracking	Record supplier purchases, auto-increment stock, track expenses	MongoDB, Counter model
Financial Analytics	Top-selling products, revenue trends, paid vs. unpaid analysis via MongoDB aggregation pipelines	MongoDB Aggregation, Redis Cache
AI Business Assistant	Chat with an LLM pre-loaded with shop's real business context (sales, stock, purchases)	Groq API (Llama 3.1-8b)

PDF Reports	Server-generated PDF sales and purchase reports emailed directly to the owner	Puppeteer, Mailjet
Profile & Shop Management	Upload profile image and payment QR code. These appear on every generated invoice	Cloudinary, Multer
Public Inventory	Every shop gets a unique public URL (jusbill.online/shop/:shopId) for customers to browse stock	React Router, nanoid

1.4 Technology Stack

Frontend

- **React 18 + Vite** — SPA with HMR during development
- **React Router DOM v6** — client-side routing with protected routes
- **Context API** — global auth & theme state (no Redux needed)
- **Framer Motion** — page transition animations
- **Tailwind CSS** — utility-first styling

Backend

- **Node.js + Express.js** — REST API server
- **Mongoose** — MongoDB ORM with schema validation
- **bcryptjs** — password hashing (cost factor 10)
- **jsonwebtoken** — stateless JWT auth stored in HttpOnly cookie
- **nanoid** — unique shopId generation
- **Puppeteer** — headless Chrome for server-side PDF generation
- **Multer + Cloudinary** — file upload pipeline

Data & Caching

- **MongoDB Atlas** — primary database (document store)
- **Upstash Redis** — serverless Redis for API response caching

External APIs

- **Groq API** — Llama 3.1-8b-instant for AI chat
- **Mailjet** — transactional email with PDF attachments
- **Cloudinary** — image CDN for profile images and QR codes

DevOps

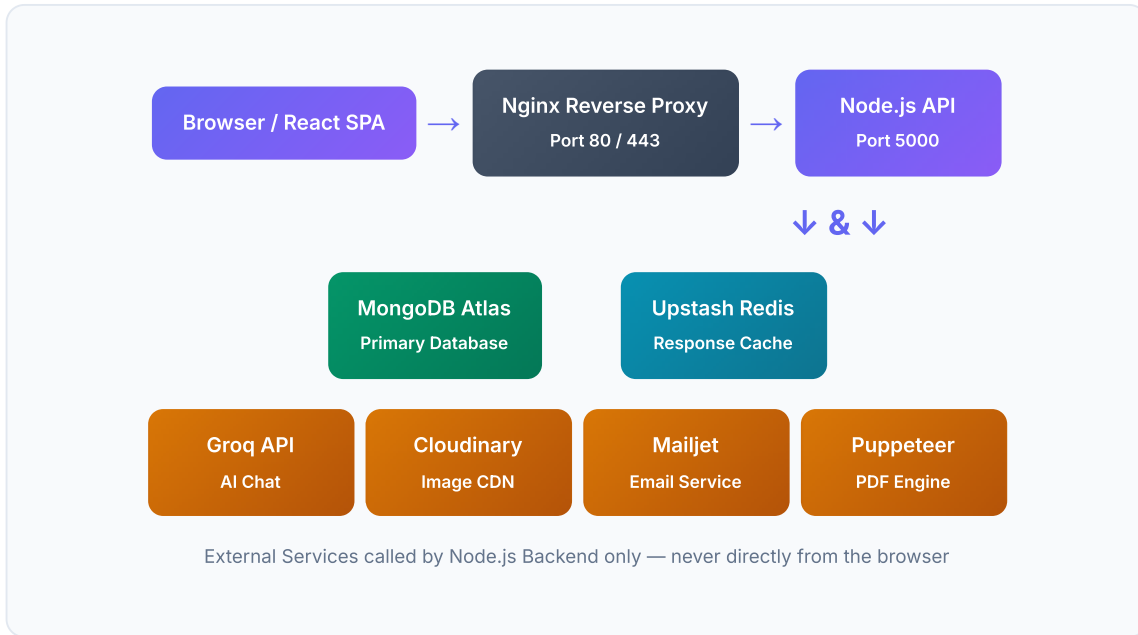
- **Docker + Docker Compose** — containerization (dev & prod profiles)
- **Nginx** — reverse proxy, SSL termination
- **Google Cloud VM** — production hosting
- **GitHub Actions** — automated DB backup workflow

2

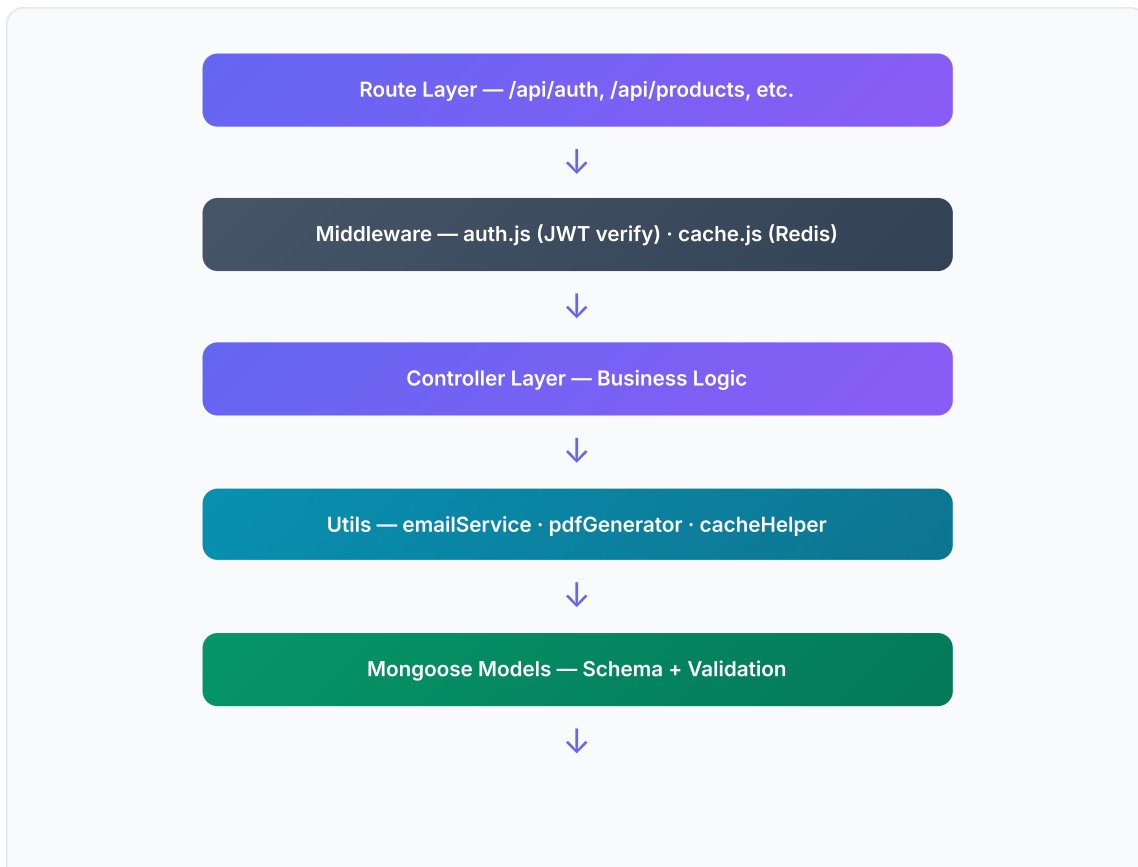
Project Architecture

High-level architecture · Layered design · Auth flow · Deployment

2.1 High-Level Architecture

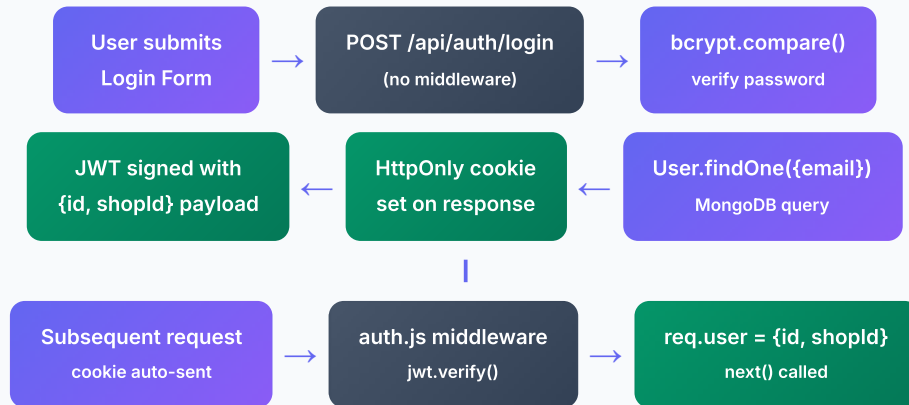


2.2 Layered Architecture (Backend)



This strict layer separation ensures each layer has a single responsibility. The middleware layer acts as a security gate. The controller contains all business logic. Models enforce data integrity at the database level.

2.3 Authentication Flow

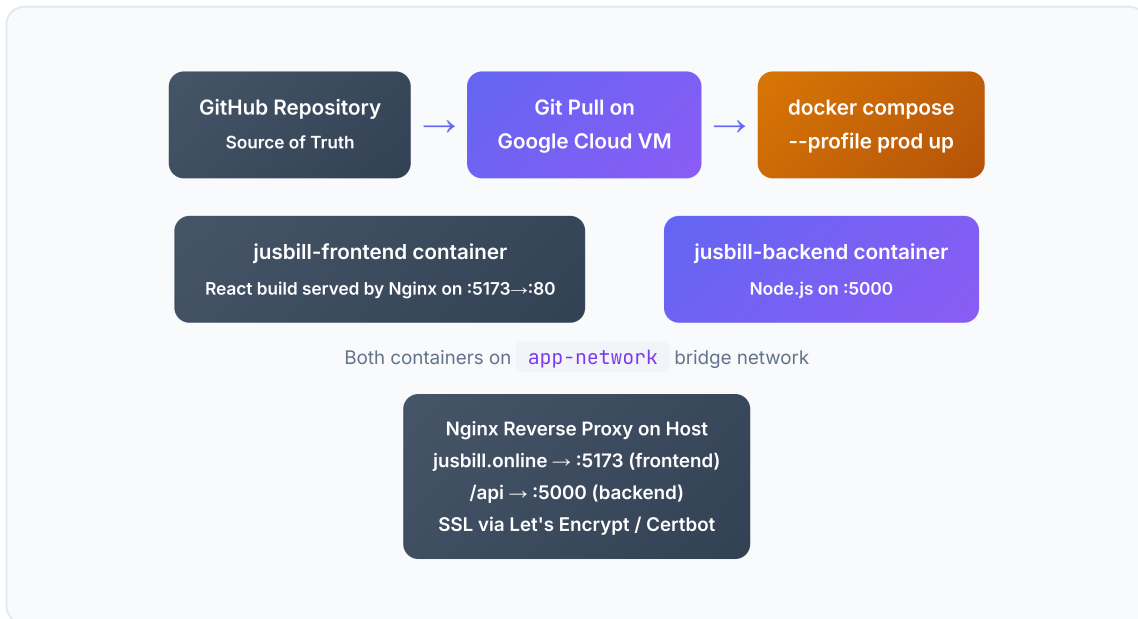


Key insight: The JWT is stored in an `HttpOnly` cookie, making it inaccessible to JavaScript. This completely prevents XSS-based token theft. The cookie is sent automatically by the browser on every request to the same domain, so no manual header injection is needed on the frontend.

2.4 Request Lifecycle

- 1 Browser** — React component fires an Axios request. Cookie is auto-attached by browser.
- 2 Nginx** — Receives the request on port 80/443. If path starts with `/api`, proxy_pass to `localhost:5000`. Otherwise, serve static React files.
- 3 Express Router** — Matches the route (e.g., `GET /api/products`) to its handler chain.
- 4 auth.js Middleware** — Reads `req.cookies.token`, calls `jwt.verify()`. Populates `req.user` with `{id, shopId}`. Calls `next()`.
- 5 Controller** — Uses `req.user.shopId` for all DB queries (tenant isolation). Checks Redis cache via `fetchWithCache()`.
- 6 Redis (Cache Hit)** — Returns cached data immediately. Response time: ~5ms. No DB query.
- 7 MongoDB (Cache Miss)** — Executes Mongoose query. Result stored in Redis with TTL. Response time: ~50–150ms.
- 8 Response** — JSON data returned to React. State updated, UI re-renders.

2.5 Deployment Architecture



The `docker-compose.yml` uses a **multi-profile setup**: `--profile dev` runs containers with volume mounts for hot-reload; `--profile prod` runs optimized production builds. The backend container has a `healthcheck` that pings `/health` every 30 seconds, and the frontend container will not start until the backend is healthy (`depends_on: condition: service_healthy`).

3.1 Multi-Tenancy via shopId

JusBill is a **multi-tenant application**. Every user gets a unique `shopId` (10-character nanoid) at registration. This `shopId` is embedded in the JWT payload and available as `req.user.shopId` in every authenticated request. Every database query for products, invoices, customers, and purchases is scoped by `shopId`, ensuring complete data isolation between shops without requiring separate databases or schemas.

Critical design decision: Instead of using the user's MongoDB `_id` as the tenant key, a separate short `shopId` was generated. This makes the shop's public inventory URL (`/shop/V8kJdQ4pRm`) short and shareable, while keeping the MongoDB ObjectId private.

3.2 Design Tradeoffs

Decision	Chosen Approach	Alternative	Why This Was Chosen
Database	MongoDB (NoSQL)	PostgreSQL (SQL)	Invoices embed line items naturally as arrays. Avoids JOIN overhead on every invoice fetch. Schema flexibility during rapid feature development.
Auth Storage	HttpOnly Cookie + JWT	localStorage + JWT	HttpOnly cookies are immune to XSS. localStorage tokens can be stolen by injected scripts. Cookie is sent automatically — no manual Authorization header needed.
PDF Generation	Puppeteer (Server-side)	jsPDF / html2canvas (Client-side)	Pixel-perfect, print-ready, text-searchable PDFs. Client-side libs render CSS inconsistently across browsers. Puppeteer uses real Chrome rendering engine.
State Management	React Context API	Redux / Zustand	Only two pieces of global state needed: auth user and theme. Adding Redux for just two contexts would be over-engineering. Context API is zero-dependency and built into React.
Caching	Upstash Redis (serverless)	Self-hosted Redis / no cache	Upstash provides a serverless REST-based Redis API with a generous free tier. No server management required. Connects from any environment including containers.
Email	Mailjet API (fetch-based)	Nodemailer / SendGrid	Nodemailer requires SMTP credentials and can be blocked by ISPs. Mailjet's HTTP API works

from any server environment including Docker containers.

AI Model	Groq (Llama 3.1-8b-instant)	OpenAI GPT-4 / Gemini	Groq offers the fastest inference speeds (~500 tokens/sec) with a generous free tier. Llama 3.1-8b is sufficient for business Q&A tasks without the cost of GPT-4.
Image Upload	Cloudinary + Multer	AWS S3 / Local disk	Cloudinary handles image transformations, CDN delivery, and storage. Local disk storage doesn't work in Docker containers (ephemeral filesystem). S3 adds more complexity.

3.3 Caching Strategy

JusBill implements a **Cache-Aside** (Lazy Loading) pattern using Upstash Redis. Cache keys are namespaced with a project prefix `jusbill_v3:` to prevent key collisions between versions.

Cache Key Pattern	TTL	Invalidated On
<code>jusbill_v3:products:{shopId}:all</code>	5 min (300s)	createProduct, updateProduct, deleteProduct, createInvoice, createPurchase
<code>jusbill_v3:invoices:{shopId}:all</code>	5 min (300s)	createInvoice, updatePaymentStatus
<code>jusbill_v3:purchases:{shopId}</code>	1 hr (3600s)	createPurchase, deletePurchase
<code>jusbill_v3:analytics:{shopId}:*</code>	10 min (600s)	createInvoice
<code>jusbill_v3:product:{id}</code>	1 hr (3600s)	updateProduct, deleteProduct

Stale Data Risk: With a 5-minute TTL, if two users are billing simultaneously from different tabs, one may see slightly outdated stock figures for up to 5 minutes. The stock *deduction* in MongoDB is always accurate because the DB write happens synchronously before the cache is invalidated.

3.4 Security Design

Threat	Mitigation in JusBill	File
XSS Token Theft	JWT in HttpOnly cookie — JS cannot read it	auth.controller.js — COOKIE_OPTIONS
Cross-Tenant Data Access	Every DB query scoped to <code>req.user.shopId</code>	All controllers
Broken Authentication	JWT verified on every protected route via auth.js middleware	middleware/auth.js

Password Breach	bcrypt hash with salt rounds=10 (~100ms/hash)	auth.controller.js line 34
Duplicate Accounts	Unique indexes on email + mobile. Error code 11000 caught and surfaced	user.js schema + auth.controller.js
Invalid GSTIN	Regex validation on backend: <code>/^[0-9]{2}[A-Z]{5}...</code>	customer.controller.js line 8
Cache Poisoning	Project-versioned key prefix <code>jusbill_v3:</code> ; cache only populated by server	config/redis.js
Stale Browser Cache (Auth)	<code>Cache-Control: no-store</code> on /me endpoint	auth.controller.js line 98

3.5 Performance Considerations

- **Compound DB indexes** on `{shopId, createdAt}` ensure invoice list queries are $O(\log n)$ even with millions of records
- **Promise.all()** in `getInvoices` runs the invoice list query and the aggregation stats query in parallel, halving the response time
- **Non-blocking cache invalidation** — after creating a product, `clearCache(...).catch(()=>{})` is called without `await`, so the HTTP response is sent immediately without waiting for Redis
- **MongoDB projection** — `getPublicProducts` uses `.select("name price stock unit description")` to avoid sending unnecessary fields over the wire
- **Background email sending** — the `sendEmailWithAttachment` call is NOT awaited; the HTTP 200 is returned immediately and email delivery happens in background
- **AI context limiting** — only last 50 invoices and 50 purchases are fetched for context, and only last 10 chat history messages are sent to Groq to control token usage

4

Database Design

7 MongoDB collections · ER diagram · Indexes · DFD

4.1 Collections Overview

Collection	Purpose	Tenant Key	Unique Constraints
users	Shop owner accounts, credentials, bank info, snapshots	N/A (top-level)	email, mobile
products	Inventory items with stock, pricing, and sales tracking	shopId	None
customers	Customer address book for invoice auto-fill	shopId	{shopId, mobile}, {shopId, email}
invoices	Sales transactions with embedded line items	shopId	{shopId, invoiceId}
purchases	Supplier purchase records with stock increment	shopId	{shopId, purchaseId}
counters	Auto-increment sequence for invoice/purchase IDs	shopId	{shopId, type}
contacts	Contact form submissions	N/A	None

4.2 Detailed Schemas

users Collection

<code>_id</code>	<code>ObjectId</code>	Auto-generated MongoDB primary key
<code>name</code>	<code>String</code>	Shop owner's full name
<code>email</code>	<code>String</code>	Login email — unique index UNIQUE
<code>password</code>	<code>String</code>	bcrypt hash (salt rounds = 10). NEVER returned in API responses
<code>shopId</code>	<code>String</code>	10-char nanoid — tenant isolation key + public shop URL param
<code>role</code>	<code>String</code>	Default: "owner" — reserved for future RBAC expansion
<code>companyName</code>	<code>String</code>	Appears on generated invoices
<code>mobile</code>	<code>String</code>	Primary contact — unique index UNIQUE
<code>gstIn</code>	<code>String</code>	GST registration number — appears on invoices

profileImage	String	Cloudinary CDN URL of uploaded profile/logo image
qrCode	String	Cloudinary CDN URL of payment QR code – embedded in PDF invoices
bankName, bankAccount, ifscCode	String	Bank details printed on invoice footer
financialSnapshot	Object	{lastSales, lastReceived, lastPending, lastSnapshotDate} – stored for period comparison on dashboard
salesSnapshot	Object	{lastSales, lastPaid, lastUnpaid, lastResetDate} – same purpose for sales-specific metric
createdAt, updatedAt	Date	Auto-managed by Mongoose timestamps: true

📁 invoices Collection

shopId	String	Tenant identifier – every query includes this filter INDEX
invoiceId	String	Human-readable ID like "INV-0042" – from Counter collection UNIQUE w/ shopId
customerId	ObjectId → Customer	Optional soft reference. Customer data also denormalized below for historical accuracy
customerName / Email / Address / Mobile / Gstin / State	String	Denormalized at invoice time so invoice is correct even if customer is later deleted
items	Array	Embedded array: [{productId, name, price, qty, total}] – snapshot of product price at time of billing
subtotal	Number	Sum of all item totals before discount
discount / discountAmount	Number	Discount % and computed discount ₹ amount
gstRate / gst	Number	GST rate % (default 18) and computed GST ₹ amount
grandTotal	Number	(subtotal - discountAmount) + gst
paymentStatus	String enum	"Paid" "Unpaid". Cannot revert from Paid to Unpaid (enforced in controller)
Indexes	Compound	{shopId, createdAt:-1} for listing · {shopId, customerName} for customer history · {shopId, invoiceId} unique

📁 products Collection

shopId	String	Tenant key INDEX
name	String	Product name for display and invoice items
price	Number	Selling price (₹) – used to compute invoice totals

purchasePrice	Number	Cost price – used by AI assistant to compute profit margin
stock	Number	Current available units. Decrementated on invoice, incremented on purchase
sold	Number	Cumulative units sold – tracked separately from stock for lifetime analytics
unit	String	Unit of measure: "kg", "pcs", "box", etc.
isRawMaterial	Boolean	If true, excluded from public inventory page. Prevents raw materials from being customer-visible.
Indexes	Compound	{shopId,name} for search · {shopId,price} for price filtering

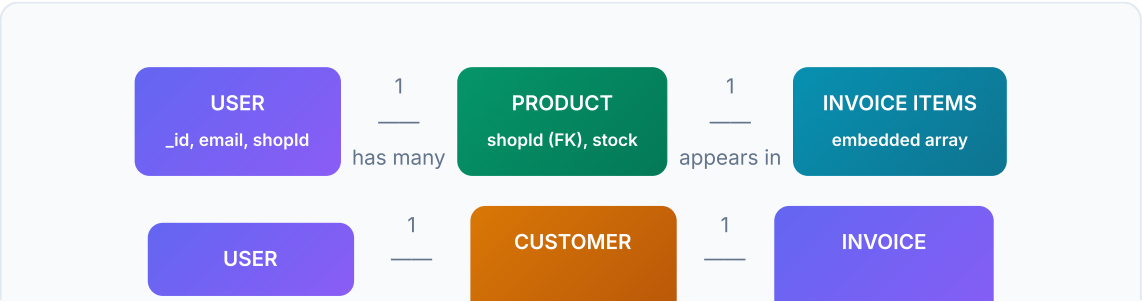
customers Collection

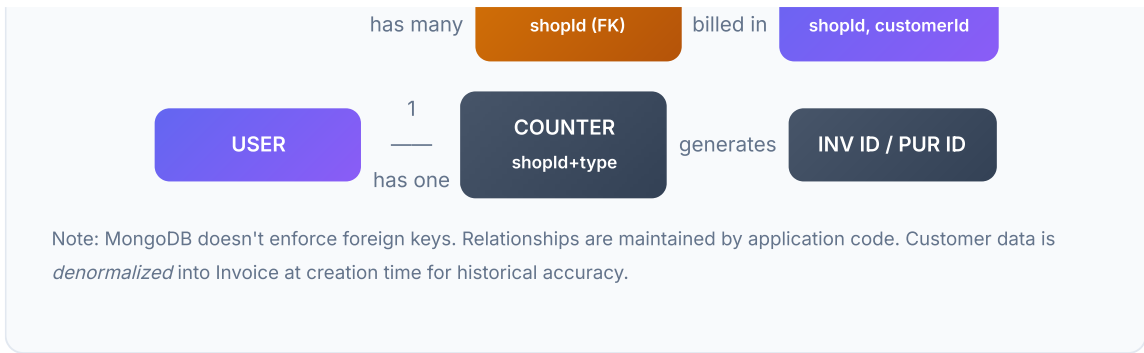
shopId	String required	Tenant key INDEX
name	String required	Customer display name
mobile	String required	Primary identifier – unique per shop UNIQUE w/ shopId
email	String required	For invoice email delivery – unique per shop UNIQUE w/ shopId
gstIn	String	Optional – regex validated: /^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]\$/
Text Index	Compound	name+mobile+email full-text for fast autocomplete on invoice form

counters Collection – Auto-increment Pattern

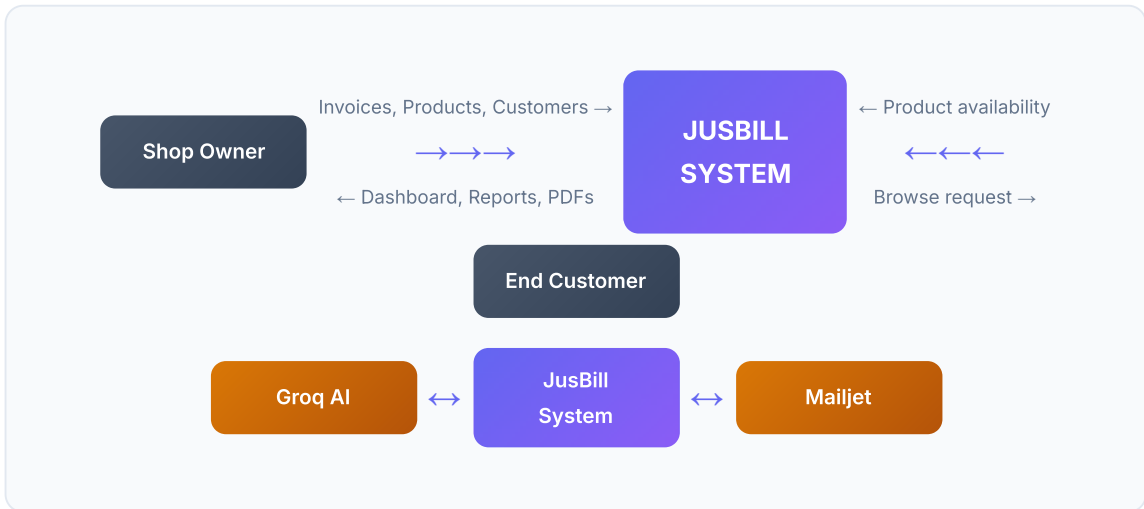
shopId	String required	Tenant key
type	String required	"invoice" or "purchase" – determines which sequence to increment
seq	Number	Monotonically increasing sequence. Atomically incremented via \$inc in findOneAndUpdate with upsert:true

4.3 ER Diagram (Conceptual)

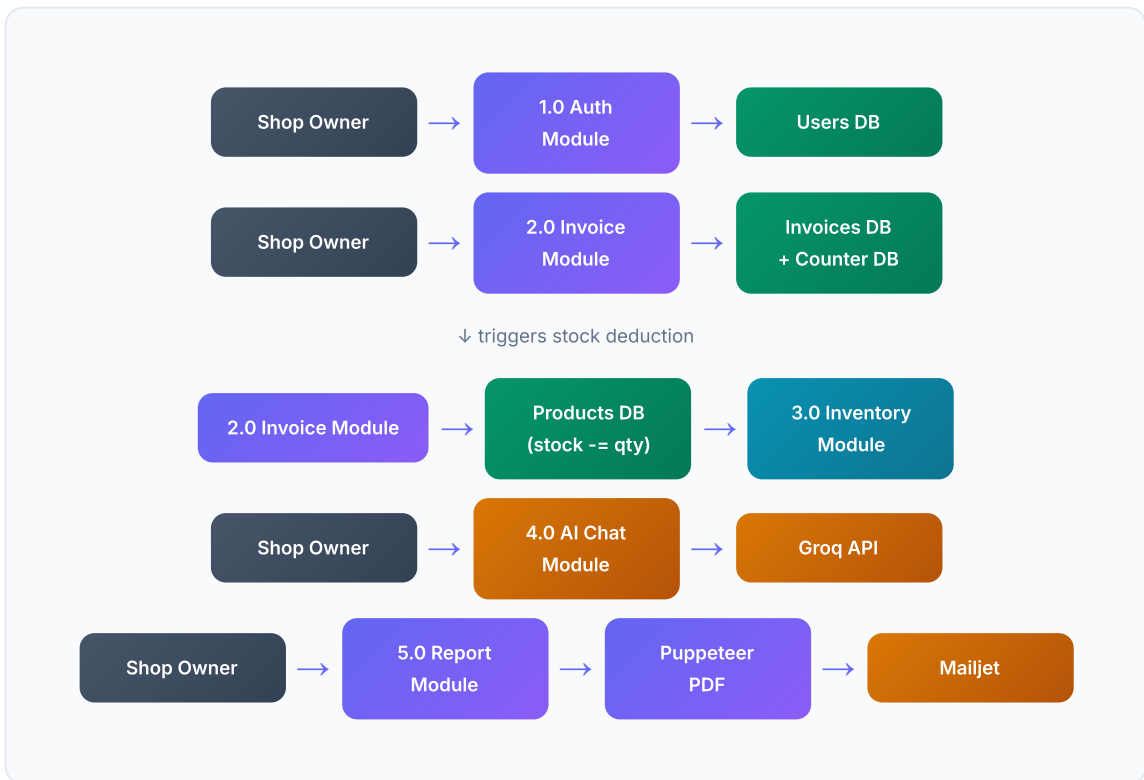




4.4 Level 0 DFD (Context Diagram)



4.5 Level 1 DFD (System Decomposition)



5

Critical Code Walkthrough

12 most important files with line-by-line explanation of critical code

5.1 middleware/auth.js — The Security Gate

backend/src/middleware/auth.js

```
const auth = async (req, res, next) => {
  const token = req.cookies.token; // Line 4: Read from HttpOnly cookie
  if (!token) return res.status(401).json({ msg: "No token" });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET); // synchronous verify
    req.user = decoded; // Line 9: { id, shopId, iat, exp }
    next(); // Pass to controller
  } catch (e) {
    res.status(401).json({ msg: "Token is not valid" }); // Expired or tampered
  }
};
```

Why this matters: This 18-line file is the single security checkpoint for all protected routes. Reading from `req.cookies.token` (not `req.headers.authorization`) is the critical design choice — it works because the frontend is on the same domain, and HttpOnly cookies cannot be read by JavaScript. The comment on line 9 (`{ id, shopId, iat, exp }`) shows exactly what the JWT payload contains — the `shopId` is the most important field for multi-tenancy.

5.2 auth.controller.js — Register & Login

backend/src/controllers/auth.controller.js

```
const COOKIE_OPTIONS = {
  httpOnly: true, // Cannot be accessed by document.cookie
  secure: false, // Set to true in production with HTTPS
  sameSite: "lax", // Allows same-site navigation, blocks CSRF cross-origin
  maxAge: 15 * 24 * 60 * 60 * 1000 // 15 days in ms
};

// In register():
const hashed = await bcrypt.hash(password, 10); // 10 salt rounds ≈ 100ms
const user = await User.create({
  name, email, password: hashed, mobile, companyName,
  shopId: nanoid(10) // "V8kJdQ4pRm" — unique, URL-safe
});
const token = jwt.sign({ id: user._id, shopId: user.shopId }, JWT_SECRET, { expiresIn: "15d" });
res.cookie("token", token, COOKIE_OPTIONS);

// Error handling for duplicate accounts:
if (error.code === 11000) {
```

```
const field = Object.keys(error.keyPattern)[0]; // "email" or "mobile"
return res.status(400).json({ msg: `${field} already exists` });
}
```

Why code 11000? MongoDB throws error code 11000 on duplicate key violations. The controller catches this specifically and extracts which field caused the violation using `error.keyPattern` — a much better user experience than a generic 500 error.

5.3 invoice.controller.js — createInvoice (Most Complex Function)

backend/src/controllers/invoice.controller.js — createInvoice()

```
for (let item of items) {
  const product = await Product.findById(item.productId); // Validate product exists
  if (product.stock < item.qty) { // Stock validation BEFORE write
    return res.status(400).json({ message: `Insufficient stock for ${product.name}` });
  }
  product.stock -= item.qty; // Deduct stock synchronously
  product.sold = (product.sold || 0) + item.qty; // Track lifetime sales
  await product.save(); // Persist immediately before moving to next item
}

// Auto-increment invoice ID using Counter collection:
const counter = await Counter.findOneAndUpdate(
  { shopId: req.user.shopId, type: "invoice" },
  { $inc: { seq: 1 } }, // Atomic increment
  { new: true, upsert: true } // Create if doesn't exist
);
const invoiceId = `INV-${String(counter.seq).padStart(4, '0')}`; // INV-0001

// Non-blocking cache invalidation (fire and forget):
clearCache(`invoices:${req.user.shopId}`).catch(() => {});
clearCache(`products:${req.user.shopId}`).catch(() => {});
clearCache(`analytics:${req.user.shopId}`).catch(() => {});
```

Three cache keys invalidated: When an invoice is created, products stock changed (so product cache must clear), invoice list changed, and analytics (top-selling) changed. All three are invalidated non-blockingly — the HTTP 201 response is returned before Redis responds.

5.4 invoice.controller.js — getInvoices with Promise.all

backend/src/controllers/invoice.controller.js — getInvoices()

```
const [invoices, [statsResult]] = await Promise.all([
  Invoice.find(query).sort({ createdAt: -1 }),
  Invoice.aggregate([
    { $match: { shopId: req.user.shopId } },
    { $group: {
      _id: null,
      totalSales: { $sum: "$grandTotal" },
      totalReceived: { $sum: { $cond: [{ $eq: ["$paymentStatus", "Paid"] }, "$grandTotal",
        totalPending: { $sum: { $cond: [{ $ne: ["$paymentStatus", "Paid"] }, "$grandTotal",
```



```

    })
  });

```

Why Promise.all here? The invoice list and the financial stats aggregation are two independent queries — they don't depend on each other's results. Running them sequentially would mean $\sim 150\text{ms} + \sim 100\text{ms} = \sim 250\text{ms}$. With `Promise.all`, both run concurrently and complete in $\sim \max(150\text{ms}, 100\text{ms}) = \sim 150\text{ms}$. A 40% faster response.

5.5 config/redis.js — Key Namespacing

```
backend/src/config/redis.js
```

```

const PROJECT_PREFIX = "jusbill_v3";
export const getKey = (key) => `${PROJECT_PREFIX}:${key}`;
// Usage: getKey("products:shopId:all") => "jusbill_v3:products:shopId:all"

```

The version prefix (`jusbill_v3`) is critical. When a breaking schema change is deployed, simply changing the prefix to `jusbill_v4` instantly invalidates ALL cached data across all shops without needing to manually flush Redis. It's a zero-downtime cache busting strategy.

5.6 utils/cacheHelper.js — Reusable Cache Pattern

```
backend/src/utils/cacheHelper.js
```

```

export const fetchWithCache = async (rawKey, ttl, fetchCallback) => {
  const cacheKey = getKey(rawKey);
  try {
    const cached = await redis.get(cacheKey);
    if (cached) return cached; // Cache hit - return immediately
  } catch (error) { /* Redis failure non-fatal - fall through to DB */ }

  const data = await fetchCallback(); // Execute the actual DB query

  try {
    await redis.set(cacheKey, data, { ex: ttl });
  } catch (error) { /* Cache set failure non-fatal */ }

  return data;
};

```

The Circuit Breaker pattern: Both the `redis.get` and `redis.set` are wrapped in try-catch. If Redis goes down, the application continues working normally using MongoDB directly. This is a critical resilience design — Redis is a performance enhancement, not a dependency.

5.7 chat.controller.js — AI Context Building

```
backend/src/controllers/chat.controller.js
```

```
const getShopContext = async (shopId) => {
  const [products, invoices, purchases] = await Promise.all([
    Product.find({ shopId }).select("name price stock unit sold purchasePrice"),
    Invoice.find({ shopId }).sort({ createdAt: -1 }).limit(50), // Last 50 only
    Purchase.find({ shopId }).sort({ createdAt: -1 }).limit(50)
  ]);
  // Build human-readable text summary for the LLM system prompt...
};

// System prompt includes real data — prevents hallucination:
const systemContent = `You are JusBill AI... Do not make up data.
SHOP DATA SUMMARY:
- Total Sales: ₹${data.totalSales}
- Products: ${data.productSummary}`;

// Last 10 messages only — controls token cost:
for (const msg of history.slice(-10)) { messages.push(...); }
```

Why backend and not frontend? If the Groq API call was made from the frontend, the API key would be exposed in browser network requests. The backend acts as a secure proxy — client sends message, backend fetches live shop data, constructs the prompt with real business context, calls Groq, and returns the response.

5.8 purchase.controller.js — Stock Reversion on Delete

backend/src/controllers/purchase.controller.js — deletePurchase()

```
// When deleting a purchase, REVERT the stock that was added:
for (const item of purchase.items) {
  if (item.productId) {
    await Product.findByIdAndUpdate(item.productId, { $inc: { stock: -item.qty } });
  }
}

// Also decrement the counter (controversial choice):
await Counter.findOneAndUpdate(
  { shopId: req.user.shopId, type: "purchase" },
  { $inc: { seq: -1 } } // WARNING: Can cause ID reuse
);
```

Interview Trap: Decrementing the counter on delete means IDs can be reused. PUR-0005 might be deleted and then reassigned to a new purchase. This is a known tradeoff — human-readable sequential IDs vs. guaranteed uniqueness. The unique index on `{shopId, purchaseId}` prevents actual duplicates at any given time, but an interviewer will definitely ask about this.

5.9 AuthContext.jsx — The Login Flash Prevention

frontend/src/context/AuthContext.jsx

```
// Initialize from localStorage to prevent white flash on refresh:
const [user, setUser] = useState(() => {
  try {
    const stored = localStorage.getItem("jusbill_user");
    return stored ? JSON.parse(stored) : null;
  } catch { return null; }
});

// Then silently verify with backend in background:
useEffect(() => { checkUser(); }, []);
// checkUser() calls /api/auth/me - if 401, clears localStorage + user state
```

The lazy initializer function in `useState(() => {...})` runs only once during component mount. It seeds the UI from localStorage immediately, preventing a login redirect flash. The background `checkUser()` then validates the cookie with the server — if the JWT expired, the state is cleared and the user is redirected to login.

5.10 middleware/cache.js — Cache Invalidation

```
backend/src/middleware/cache.js
```

```
export const clearCache = async (pattern) => {
  const fullPattern = getKey(`${pattern}*`); // Wildcard suffix
  const keys = await redis.keys(fullPattern); // Scan all matching keys
  if (keys.length > 0) await redis.del(...keys); // Batch delete
};
```

The `clearCache("invoices:shopId123")` call deletes ALL keys matching `jusbill_v3:invoices:shopId123*`. This is important because search queries generate keys like `invoices:shopId123:rice`, `invoices:shopId123:sugar`. The wildcard ensures ALL search-specific caches are cleared, not just the "all" key.

5.11 config/upload.js — Multer + Cloudinary Pipeline

```
backend/src/config/upload.js
```

```
const storage = new CloudinaryStorage({
  cloudinary: cloudinary,
  params: {
    folder: "jusbill_profiles", // Organized in Cloudinary
    allowed_formats: ["jpg", "png", "jpeg"], // Whitelist file types
  },
});
export const upload = multer({ storage: storage });
```

The `CloudinaryStorage` adapter intercepts Multer's normal disk write, instead streaming the file directly to Cloudinary's servers. The file never touches the server's local disk — critical for Docker containers where the filesystem is ephemeral (data lost on container restart).

5.12 product.controller.js — getPublicProducts

backend/src/controllers/product.controller.js

```
export const getPublicProducts = async (req, res) => {
  const { shopId } = req.params;
  const shop = await User.findOne({ shopId }).select("companyName");
  const products = await Product
    .find({ shopId, isRawMaterial: { $ne: true } }) // Exclude raw materials
    .select("name price stock unit description") // No sensitive fields
    .sort({ createdAt: -1 });
  res.json({ shopName, products });
};
```

NO auth middleware on this route. This is intentional — it's a public endpoint. The `isRawMaterial: { $ne: true }` filter is critical: it hides raw materials (like supplier pricing) from end customers. The `.select()` projection ensures sensitive fields like `purchasePrice` (cost price) and `sold` are never exposed publicly.

6

Complete Feature Flows

Every major feature from user action to database and back

6.1 User Registration Flow

Files: RegisterPage.jsx → auth.api.js → auth.routes.js → auth.controller.js → user.js

- 1 **User Action:** Fills name, email, password, mobile, company name on RegisterPage.jsx. Clicks "Register".
- 2 **Frontend:** AuthContext. `register()` calls `registerApi(data)` — Axios POST to `/api/auth/register`. No auth header needed.
- 3 **Express Router:** `router.post("/register", register)` — no `auth` middleware because the user doesn't have a token yet.
- 4 **Controller Validates:** Email regex, password length ≥ 8 , all required fields present.
- 5 **Password Hashing:** `bcrypt.hash(password, 10)` — generates a salted hash. Cost factor 10 $\approx$ 100ms intentionally slow to resist brute force.
- 6 **MongoDB Write:** `User.create({...password: hashed, shopId: nanoid(10)})`. If duplicate email/mobile, MongoDB throws error code 11000.
- 7 **JWT Signed:** `jwt.sign({id, shopId}, JWT_SECRET, {expiresIn: "15d"})`. Payload contains only `id` and `shopId` — minimum required data.
- 8 **Cookie Set:** `res.cookie("token", token, {httpOnly:true, sameSite:"lax", maxAge: 15days})`. Browser stores this automatically.
- 9 **Frontend State:** AuthContext sets `user` state and writes to `localStorage`. React Router navigates to `/dashboard`.

6.2 Invoice Creation Flow (Most Complex)

Files: InvoicePage.jsx → invoice.api.js → invoice.routes.js → auth.js → invoice.controller.js → product.js + counter.js + invoice.js + cache.js

- 1 **User Action:** Selects customer, adds products with quantities, sets discount and GST rate, clicks "Create Invoice".
- 2 **auth.js Middleware:** Reads JWT from cookie, decodes `{id, shopId}`, attaches to `req.user`. Calls `next()`.
- 3 **Stock Validation Loop:** For each item, fetches product from MongoDB. If `product.stock < item.qty`, returns 400 with product name and available quantity.
- 4 **Atomic Stock Deduction:** `product.stock -= item.qty` then `product.sold += item.qty` then `product.save()`. Sequential saves per item ensure no partial updates.
- 5 **Price Calculation:** `subtotal =  $\Sigma(\text{price} \times \text{qty})$  → discountAmount = subtotal × (discount/100) → gst = (subtotal - discountAmount) × (gstRate/100) → grandTotal = subtotal - discountAmount + gst`

- 6 **Invoice ID Generation:** `Counter.findOneAndUpdate({shopId, type:"invoice"}, {$inc:{seq:1}}, {upsert:true})` atomically returns next sequence. Formatted as `INV-0042` .
- 7 **Invoice Saved:** `Invoice.create({shopId, invoiceId, customerName/Email/Address/Mobile/Gstin/State, items:finalItems, subtotal, discount, gst, grandTotal})` . Customer data denormalized for historical accuracy.
- 8 **Cache Invalidated (non-blocking):** `clearCache("invoices:shopId").catch(()=>{})` , `clearCache("products:shopId").catch(()=>{})` , `clearCache("analytics:shopId").catch(()=>{})` .
- 9 **Response:** 201 with invoice object. Frontend generates PDF from invoice data using the invoice HTML template and offers download/share options.

6.3 AI Chat Flow

Files: ChatPage.jsx → chat.api.js → chat.routes.js → auth.js → chat.controller.js → Groq API

- 1 **User Action:** Types "What is my best selling product?" in ChatPage.jsx. Frontend sends `{message, history: [...last10msgs]}` .
- 2 **Context Fetching:** `getShopContext(req.user.shopId)` uses `Promise.all()` to simultaneously fetch all products, last 50 invoices, last 50 purchases.
- 3 **System Prompt Built:** Business data converted to human-readable text and injected into system prompt. The AI is instructed to ONLY answer based on this data and not make up information.
- 4 **History Appended:** Last 10 messages from `history` array added to messages array for conversational context. New user message added last.
- 5 **Groq API Called:** POST to `https://api.groq.com/openai/v1/chat/completions` with model `llama-3.1-8b-instant` , max_tokens 1024, temperature 0.7, stream: false.
- 6 **Response:** `result.choices[0].message.content` returned to frontend. ChatPage displays it and appends to conversation history in local state.

6.4 PDF Generation + Email Flow

Files: invoice.controller.js → pdfGenerator.js → emailService.js → Mailjet API

- 1 **User Action:** Clicks "Email Invoice" on InvoicePage. Frontend generates PDF as base64 string, sends `{email, customerName, invoiceId, pdfBuffer (base64), type:"Invoice"}` to `POST /api/invoice/send-email` .
- 2 **Controller:** Converts base64 back to Buffer: `Buffer.from(pdfBuffer, 'base64')` .
- 3 **Background Email:** `sendEmailWithAttachment(...).catch(err => console.error(err))` — NOT awaited. The 200 response is sent to the client immediately.
- 4 **Mailjet API:** emailService.js encodes credentials as base64 Basic auth, sends PDF as `Base64Content` in Mailjet's Messages format. ContentType: "application/pdf".
- 5 **Sales Report:** For reports, Puppeteer generates the PDF on the server (`generateSalesReportPdf`), then that buffer is passed to `sendEmailWithAttachment`. Same background pattern.

6.5 Profile Image Upload Flow

Files: ProfilePage.jsx → user.routes.js → upload.js (Multer+Cloudinary) → user.controller.js

- 1 **User Action:** Selects image file in ProfilePage. Frontend uses FormData to send multipart/form-data with file attached.
- 2 **Route middleware:** `router.put("/profile", auth, upload.fields([name:"profileImage"], {name:"qrCode"}]), updateProfile)` . The `upload` middleware runs before the controller.
- 3 **Multer + Cloudinary:** CloudinaryStorage intercepts the file stream, uploads directly to Cloudinary CDN, returns the public URL. File never touches the server disk.
- 4 **Controller:** `req.files.profileImage[0].path` contains the Cloudinary URL. `User.findByIdAndUpdate()` saves this URL to the profileImage field.
- 5 **Cache Invalidation:** `redis.del(getKey("user:{id}"))` clears the user cache so next auth verify picks up the new profile image URL.

6.6 Purchase Entry + Stock Increment Flow

Files: ProductPage.jsx (purchase form) → purchase.controller.js → product.js (stock) + counter.js

- 1 **User Action:** Records a purchase from supplier — selects products, enters purchase price and quantity.
- 2 **Counter:** Atomically increments purchase counter. Generates ID like `PUR-0012` .
- 3 **Purchase Saved:** `Purchase.create({shopId, purchaseId, supplierName, items, totalAmount})` .
- 4 **Stock Increment:** For each item with a `productId` , `Product.findByIdAndUpdate({$inc: {stock:qty}, $set:{purchasePrice}})` . Also updates the cost price.
- 5 **Cache Cleared:** Both `purchases:shopId` and `products:shopId` caches invalidated.

7

Project-Specific Interview Questions

25 questions — every one tied to actual code in the repository

1

I see in `auth.js` middleware you read the token from

`req.cookies.token`

. Why cookies and not the Authorization header? What are the security implications?

Answer: Storing the JWT in an `HttpOnly` cookie means JavaScript running in the browser — including any injected malicious script (XSS) — cannot read the token via `document.cookie`. If I used `localStorage` and sent it as `Authorization: Bearer token`, any XSS attack could steal the token and use it from any origin. The `HttpOnly` flag completely closes that attack vector. The `sameSite: "lax"` setting also provides CSRF protection by only sending the cookie on same-site navigations. The tradeoff is that cookie-based auth doesn't work for native mobile apps or third-party API consumers — but for a browser SPA on the same domain, it's the superior choice.

File: `middleware/auth.js (line 4)` + `auth.controller.js (COOKIE_OPTIONS)`

↳ Follow-up: "What happens if the cookie is set with `secure: false` in production?" — Answer: The cookie would be sent over plain HTTP, making it readable by a man-in-the-middle attacker. I have a comment in the code noting this needs to be reverted to `secure: isProd` for HTTPS production use.

2

In your

`createInvoice`

function, you loop through items sequentially using a

`for...of`

loop and

`await`

inside. Why not use

`Promise.all`

here for parallel processing?

Answer: This is an intentional design choice for correctness over speed. If I used `Promise.all`, all product stock reads would happen simultaneously, before any writes. If someone buys the last unit of "Rice" and the last unit of "Sugar" in the same invoice, both reads would see stock > 0 and both writes would proceed — but with two concurrent invoices, you could oversell. The sequential `for...of` `await` ensures each product's stock is read, validated, and decremented atomically before moving to the next product. Correctness matters more than the ~50ms saved by parallelization in this financial context.

File: `invoice.controller.js (lines 19-43)`

↳ Follow-up: "Then why did you use `Promise.all` in `getInvoices`?" — Because there, the two queries are reads with no write dependencies between them — parallel reads are perfectly safe.

3

I notice your

`findOneAndUpdate`

with

`$inc`

and

`upsert: true`

. Why this pattern for auto-incrementing invoice IDs?

Counter model uses

Why not use MongoDB's auto-generated ObjectId?

Answer: ObjectIds are 24-character hex strings like `507f1f77bcf86cd799439011` — not human-friendly for Indian shop owners. They need to look at an invoice number and immediately recognize `INV-0042` in their GST records. The `findOneAndUpdate` with `$inc` is MongoDB's atomic counter pattern — the increment operation is guaranteed to be atomic at the document level, so even under concurrent invoice creation, two invoices will never get the same sequence number. The `upsert: true` creates the counter document the first time automatically.

File: `models/counter.js` + `invoice.controller.js` (lines 51-57)

↳ Follow-up: "What if two requests hit `createInvoice` simultaneously — can they get the same `invoiceId`?" — No. MongoDB's `$inc` on a single document is atomic. The second request will always see the incremented value from the first.

4 In `getInvoices`, `Invoice.find()` and `Invoice.aggregate()` inside `Promise.all()` you call

Answer: The aggregation pipeline has two stages. First, `$match: {shopId: req.user.shopId}` filters to only this shop's invoices — equivalent to a SQL WHERE clause. Second, `$group: {_id: null}` groups ALL matched documents into a single bucket (null means one group for all). Inside the group, `$sum: "$grandTotal"` sums all grand totals for `totalSales`. For `totalReceived`, I use `$cond: [{ $eq: ["$paymentStatus", "Paid"] }, "$grandTotal", 0]` — a conditional expression that includes the `grandTotal` only if `paymentStatus` is "Paid", otherwise contributes 0. `totalPending` does the inverse with `$ne`. This gives the dashboard stats in a single DB round-trip instead of three separate queries.

File: `invoice.controller.js` (lines 104-121)

↳ Follow-up: "Why is the aggregation result destructured as `[statsResult]` in array notation?" — Because `aggregate()` always returns an array. With `_id: null` there's either 0 or 1 result. Destructuring `[statsResult]` gets the first element, which could be undefined if there are no invoices — handled by the null check on line 123.

5 I see a `getKey()` function in `redis.js` that prefixes every cache key with `"jusbill_v3:"`. What problem does this versioned prefix solve?

Answer: It solves the cache invalidation problem on deployments. Suppose I change the invoice schema — maybe add a new field or rename one. If I deploy the new code and the old cached data (without the new field) is still in Redis, the frontend would receive stale, incomplete data and potentially crash. By changing the prefix from `jusbill_v3` to `jusbill_v4`, every single cache key across all shops is effectively invalidated instantly on the next deployment — without touching Redis directly, without

running any scripts, and without any downtime. Old keys still exist but they'll never be read by the new code. Redis will automatically evict them based on memory pressure.

File: `config/redis.js` (line 15)

↳ Follow-up: "What happens if Redis is down entirely?" — The `cacheHelper.js` catches all Redis errors and falls through to MongoDB. The app continues working normally, just without caching performance benefits. Redis is a performance layer, not a critical dependency.

6 In `invoice.controller.js`, you store `customerName`, `customerEmail`, `customerAddress` and other `customer` fields directly on the `Invoice` document even though you already have a `customerId` reference. Why this denormalization?

Answer: This is a deliberate denormalization for historical accuracy. If a customer updates their address after an invoice is created, we don't want the old invoice to show the new address — that would be incorrect for GST compliance purposes. By snapshotting customer data at invoice creation time, each invoice is a complete, immutable historical record. Even if the customer is deleted, the invoice still has all their details. In SQL, you'd typically use a snapshot table or just accept stale data. In MongoDB, embedding the data in the document is natural and efficient since there's no JOIN penalty.

File: `models/invoice.js` (lines 8-13) + `invoice.controller.js` (lines 63-68)

↳ Follow-up: "Then what's the purpose of the `customerId` ObjectId reference?" — It's kept for potential future features like customer invoice history — querying all invoices for a specific customer by their ID. Currently it's populated but not heavily used in queries.

7 Your `getShopContext` function in `chat.controller.js` fetches data with `.limit(50)`. Why 50? What happens if a shop has 10,000 invoices?

Answer: The limit of 50 is a token budget constraint. Llama 3.1-8b-instant has a context window, and the system prompt already uses significant tokens. Sending 10,000 invoice summaries would exceed the model's context window and Groq's rate limits. The last 50 invoices are the most business-relevant for current-state questions like "what's my recent revenue?" For historical queries, the aggregated totals (`totalSales`, `totalPaid`, `totalUnpaid`) are calculated from ALL invoices — so the AI has accurate lifetime totals while getting recent detail for trend analysis.

File: `chat.controller.js` (lines 12-13)

↳ Follow-up: "What's the temperature: 0.7 setting doing in the Groq API call?" — Temperature controls randomness. 0.0 is deterministic (same input → same output). 1.0 is highly random. 0.7 gives balanced responses — factual but not robotic. For a business assistant giving financial data, I wouldn't go above 0.7.

8

In `ProtectedRoute.jsx`

, there's this line:

```
if (loading) return
children ? children
: <Outlet />
```

. Why return the children/Outlet instead of a loading spinner during the auth check?

Answer: Because of how we initialize AuthContext. The user state is seeded immediately from localStorage — so on page reload, `user` is not null and `loading` is true simultaneously. If ProtectedRoute returned null or a spinner while loading, the user would see a flash of blank screen or spinner before the dashboard appeared. By returning the children directly while loading is true AND we have a cached user, the dashboard renders immediately. The background `verify()` call then silently validates with the server. If the token is invalid (expired), `verify()` fails, sets user to null, and React re-renders — automatically redirecting to login.

File: `components/ProtectedRoute.jsx (line 8)` + `context/AuthContext.jsx`

↳ Follow-up: "Isn't there a security risk — the dashboard renders briefly even if the token is expired?" — The dashboard renders, but all API calls immediately get 401 responses because the server validates the JWT independently. No actual data loads. The UI might flash briefly before redirecting.

9

In `auth.controller.js`

line 98, you have

```
res.setHeader("Cache-Control", "no-store")
```

on the

`/api/auth/me`

endpoint. There's even a comment explaining why. What exact bug does this prevent?

Answer: Without this header, browsers can return a cached 304 Not Modified response for the `/me` endpoint. The 304 response doesn't re-send headers including CORS headers. When a user switches from the HTTPS domain to the plain HTTP IP for testing, or when the origin changes slightly, the cached 304 response from the old origin comes back without the correct CORS headers. The browser's CORS check fails, the request is blocked, and from the frontend's perspective it looks like a 401 — silently logging the user out. Setting `no-store` forces every `/me` request to go to the server and return a fresh response with current CORS headers.

File: `auth.controller.js (lines 96-98)`

↳ Follow-up: "Doesn't this hurt performance?" — Yes, but `/me` is called only once on app load, not on every page navigation. The performance cost is negligible compared to the bug it prevents.

10

Your `updatePaymentStatus`

endpoint explicitly rejects changing status from "Paid" back to "Unpaid". Why is this business rule enforced at the backend and not just the frontend?

Answer: Because frontend validation is cosmetic — any user with Postman or curl can call the API directly bypassing the UI. Financial records require backend enforcement. Once an invoice is marked as

Paid, allowing it to revert to Unpaid could be used to manipulate accounting records — for example, marking a paid invoice as unpaid to show lower income for GST returns. Enforcing this at the database/controller layer makes it tamper-proof regardless of how the API is accessed. The enum constraint on the schema `["Paid", "Unpaid"]` also prevents any other status values from being stored.

File: `invoice.controller.js` (lines 148-150) + `models/invoice.js` (line 29)

↳ Follow-up: "What if the shop owner marks an invoice as Paid accidentally?" — Current design intentionally has no revert. A future improvement could be a dedicated admin undo action with an audit trail, rather than a simple toggle.

11 Why does `getPublicProducts` use `isRawMaterial: {$ne: true}` instead of `isRawMaterial: false` ? What's the difference?

Answer: This is a subtle but important MongoDB query difference. `{isRawMaterial: false}` only matches documents where the field is explicitly set to `false`. But many older product documents created before the `isRawMaterial` field was added might not have this field at all — the field would be `undefined`, not `false`. `{$ne: true}` matches documents where `isRawMaterial` is anything other than `true` — including `false`, `undefined`, or `null`. This makes the query backwards-compatible with products created before the field existed, without needing a database migration.

File: `product.controller.js` (line 136)

↳ Follow-up: "How would you fix this properly if you had time?" — A database migration script that sets `isRawMaterial: false` on all existing products. Then the query can use `isRawMaterial: false` reliably. Or define a Mongoose default of `false` — already done on line 13 of `product.js`, so only legacy pre-field documents are affected.

12 In `pdfGenerator.js`, I see you launch Puppeteer with `args: ["--no-sandbox", "--disable-setuid-sandbox"]`. Why are these flags needed? What are the security implications?

Answer: Puppeteer's sandboxing requires certain Linux kernel features (namespaces) that Docker containers don't enable by default. Without `--no-sandbox`, Puppeteer crashes immediately inside a container with a "Running as root without --no-sandbox is not supported" error. The sandbox normally provides an extra isolation layer where Chrome's renderer process runs in a separate restricted environment. Disabling it means all Chromium processes run in the same process space. The security risk is that if a malicious website were being rendered — but in JusBill, Puppeteer only renders our own HTML strings constructed by the server, never user-supplied URLs. So the attack surface is minimal.

File: `utils/pdfGenerator.js` (lines 62-66)

↳ Follow-up: "Why do you launch a new browser on every PDF request instead of reusing one instance?" — Current implementation (for simplicity) launches and closes per request. For high load, this should be improved to a browser pool pattern — one persistent browser, multiple pages opened concurrently.

13 Why did you choose Context API over Redux or Zustand for state management in this project?

Answer: JusBill only has two pieces of global state: the authenticated user object and the theme (dark/light). Context API handles both with zero dependencies and zero boilerplate. Redux would add a store, actions, reducers, selectors, and potentially thunks or sagas — all for managing two variables. The complexity isn't justified. Context API has a known performance weakness with frequent updates (every context consumer re-renders), but since auth state changes only on login/logout/register — not on every keystroke — this limitation doesn't apply here. If the app grew to include complex real-time state (cart, notifications, WebSocket messages), Redux or Zustand would be worth adding.

File: `context/AuthContext.jsx` + `context/ThemeContext.jsx`

Follow-up: "Why not Zustand specifically?" — Zustand is an excellent choice and would be my first upgrade if state complexity grew. The key advantage of Zustand over Context is that it uses selector functions to prevent unnecessary re-renders. For this project's current scope, Context API was sufficient.

14 The `docker-compose.yml` has a `healthcheck` that tests the `/health` endpoint, and the frontend depends on the backend with

Answer: Without the healthcheck dependency, Docker starts both containers nearly simultaneously. The frontend Nginx container starts, but the Node.js backend might still be connecting to MongoDB (which can take 3-5 seconds on cold start). If a user hits the frontend immediately, API requests to the backend fail with connection refused errors. With `condition: service_healthy`, Docker Compose waits until the backend's `/health` endpoint returns 200 before starting the frontend. The `/health` endpoint returns immediately once the Express server is ready — and since `connectDB()` is called before `app.listen`, MongoDB is also connected by that point.

File: `docker-compose.yml` (lines 53-58, 126-128) + `backend/src/app.js` (line 32-34)

Follow-up: "What does the `/health` endpoint return?" — `{status: "ok", timestamp: new Date().toISOString()}`. Simple enough for the Docker health probe but also useful for external uptime monitoring services.

15 In `emailService.js`, you build Mailjet auth using `Buffer.from(`${key}:${secret}`).toString('base64')` instead of using Mailjet SDK. Why?

Answer: Mailjet's v3.1 API uses HTTP Basic Authentication with the API key and secret key. Base64-encoding `apiKey:secretKey` is exactly how Basic Auth works per RFC 7617. Using the native `fetch` API (available natively in Node 18+) means zero additional dependencies. The Mailjet SDK would add a

package dependency, increase container image size, and abstract away the simple HTTP call. For a single POST request with a fixed structure, a direct fetch call is more transparent, easier to debug, and has no version compatibility concerns. If the Mailjet API changes a field name, I can see and fix it directly in my code.

File: `utils/emailService.js` (lines 18-51)

↳ Follow-up: "What happens when Mailjet credentials are missing?" — Line 11-14: the function returns early with a `console.warn` but doesn't throw. This means the app continues working in development without email credentials configured — emails just silently skip.

16 Your text on `{name: 'text', mobile: 'text', email: 'text'}`. But `getCustomers` query `$regex`, `$text`. Why have the text index at all?

Customer index collection has a

Answer: This is actually a technical inconsistency in the codebase. The text index was added with the intention of using MongoDB's `$text` operator for fast full-text search. However, the controller currently uses `$regex` with case-insensitive flag instead. The `$regex` approach doesn't use the text index and is effectively a collection scan on large datasets. A proper improvement would be to replace `$regex` with `$text: {$search: searchTerm}` which uses the text index and supports partial word matching. This is a known technical debt — a good interviewer will catch it and it shows self-awareness to acknowledge it.

File: `models/customer.js` (line 16) + `customer.controller.js` (lines 29-32)

↳ Follow-up: "How would you fix this?" — Change the query to: `if (search) query.$text = {$search: search}`. This uses the text index and is significantly faster on large customer lists.

17 In `app.js` line 24, I see `const allowedOrigins = ["*"]` with a comment saying "Allow all origins". Why is CORS allowing all origins in what appears to be production code?

Answer: This is a known issue left from a debugging session for EC2/VM HTTP testing. When the domain was being tested over plain HTTP, CORS was causing issues with the specific origin being blocked. The wildcard was a temporary fix that wasn't reverted. In production with HTTPS and a real domain, this should be changed to `allowedOrigins = [process.env.ALLOWED_ORIGINS, "http://localhost:5173"]` — which is actually what the `docker-compose.yml` sets via the `ALLOWED_ORIGINS` environment variable. The wildcard effectively disables CORS protection — any website can make API calls to the backend while users are logged in.

File: `backend/src/app.js` (lines 24-30)

↳ Follow-up: "This combined with `HttpOnly` cookies — is it still safe?" — Partially. `HttpOnly` cookies prevent XSS token theft, but with wildcard CORS, a malicious site could make credentialed requests (with cookies) to the API.

The `sameSite: lax` cookie setting provides some cross-site protection — it blocks cookies on cross-origin POST requests from third-party sites in most cases. But this should still be fixed.

18 Walk me through exactly `connectDB` when the MongoDB connection times out. What are these three timeout values for?

Answer: Three different timeouts protect against different failure modes. `connectTimeoutMS: 20000` — how long to wait when initially establishing the TCP connection to MongoDB Atlas. If the network is slow or Atlas is down, don't hang forever. `socketTimeoutMS: 45000` — after a connection is established, how long to wait for a query response before considering the socket dead. A long-running aggregation on millions of documents might need more time. `serverSelectionTimeoutMS: 20000` — how long the Mongoose driver tries to find an available MongoDB server in the replica set before throwing "Server selection timed out". If all Atlas nodes are busy, this bounds the wait. If any of these timeouts trigger, the `catch` block logs the error and calls `process.exit(1)` — the container exits and Docker restarts it (because of `restart: always` in `docker-compose.yml`).

File: `config/db.js` (lines 5-9) + `docker-compose.yml` (line 96)

↳ Follow-up: "Why `process.exit(1)` instead of retrying?" — In production with Docker, `process.exit(1)` + `restart:always` is effectively a retry with a clean state. It's simpler than implementing exponential backoff retry logic manually.

19 In `getTopSellingProducts`, the aggregation uses `$unwind: "$items"` on the `invoices` collection. What does this do and why is it needed for this analytics query?

Answer: The Invoice document has an embedded `items` array — each invoice can have multiple products. `$unwind` deconstructs this array, creating one document per array element. So an invoice with 3 items becomes 3 documents, each with the invoice's fields plus one item's data. After unwinding, the `$group` stage can then group by `$items.productId` and `$sum: "$items.qty"` across all invoices — giving us total quantity sold per product. Without `$unwind`, you can only aggregate at the invoice level, not at the individual product level.

File: `product.controller.js` (lines 171-183)

↳ Follow-up: "What's the performance impact of `$unwind` on a large dataset?" — `$unwind` with a preceding `$match` (filtering by `shopId`) is efficient if there's an index on `shopId`. The index reduces the documents before unwinding. Without the index, `$unwind` on millions of invoice documents would be very slow.

20 Your `deletePurchase` controller decrements the counter (`$inc: {seq: -1}`) when a purchase is deleted. What's the problem with this approach?

Answer: It creates the possibility of ID reuse. If PUR-0005 is deleted and counter decrements to 4, the next purchase gets PUR-0005 again. This can cause confusion in accounting records if someone refers to the old PUR-0005. A better approach would be to never decrement the counter — deleted records leave gaps (PUR-0001, PUR-0002, PUR-0004, PUR-0005) which is perfectly acceptable and even

standard practice in financial systems. Sequential numbering without gaps is less important than uniqueness. I would remove that counter decrement in production.

📁 File: `purchase.controller.js` (lines 81-84)

↳ Follow-up: "Why was the decrement added in the first place?" — Likely for aesthetic reasons — to keep invoice IDs sequential without gaps. But this trades correctness for aesthetics, which is the wrong tradeoff in financial software.

21 The `/api/` to the `/api` with a trailing slash `GET /api/products` — what happens at each layer?
Nginx config proxies the Express app mounts all routes at handling. Walk me through a complete request: browser requests

Answer: 1) Browser sends `GET /api/products` with `HttpOnly` cookie attached automatically. 2) Nginx matches the location block `/api/`, strips or passes the path, and `proxy_passes` to `http://localhost:5000`. The request becomes `GET /api/products` on the Node server. 3) Express in `app.js` sees `app.use("/api", apiRouter)` — matches `/api` prefix, strips it, passes `/products` to `apiRouter`. 4) `apiRouter` matches `apiRouter.use("/products", productRoutes)` — passes to `productRoutes`. 5) `productRoutes` matches `router.get("/", auth, getProducts)` for the root `/`. 6) `auth` middleware runs — reads cookie, verifies JWT, sets `req.user`. 7) `getProducts` controller runs — checks Redis, then MongoDB, returns JSON.

📁 File: `backend/src/app.js` (lines 36-46) + Nginx config + `routes/product.routes.js`

↳ Follow-up: "How does Nginx know when to serve the React app vs. proxy to the backend?" — The Nginx config has two location blocks: `location /api/ { proxy_pass ... }` and `location / { try_files $uri $uri/ /index.html; }`. API requests match the more specific `/api/` block first.

22 In `user.controller.js`, the `updateProfile` function updates the user with `User.findByIdAndUpdate(req.user, updates, {new: true})`

Answer: This is a Mass Assignment vulnerability. Since `updates = req.body` without any filtering, a malicious user could POST `{role: "admin", shopId: "someotherid", password: "hackme"}` and update those privileged fields. The `password` field is especially dangerous — it would store a plaintext password, bypassing `bcrypt` entirely. The fix is to whitelist allowed fields: `const {name, mobile, companyName, address, state, gstin, bankName, bankAccount, ifscCode, secondaryMobile} = req.body; const updates = {name, mobile, ...}`. Only these fields should ever be user-modifiable through the profile endpoint.

📁 File: `user.controller.js` (lines 6-12)

↳ Follow-up: "Does Mongoose protect against this automatically?" — No. Mongoose doesn't prevent updating arbitrary fields. You'd need to manually whitelist fields, or use Mongoose's `select: false` on sensitive fields

combined with proper update filtering.

23 Why does JusBill use Upstash Redis specifically over a self-hosted Redis container or ElastiCache? How does Upstash's REST API model differ from traditional Redis?

Answer: Traditional Redis uses a persistent TCP connection with the Redis Serialization Protocol (RESP). This requires keeping a connection pool alive and doesn't work well in serverless environments where the process can be cold-started. Upstash provides a REST HTTP API — every Redis command is an HTTP request. This means it works from any environment: containers, serverless functions, even browser edge workers. No connection pooling needed. For JusBill running in a Docker container on a VM, the @upstash/redis SDK abstracts this into the same get/set/del interface. The tradeoff is slightly higher latency per command (HTTP overhead vs. persistent TCP) — but since we're doing single key reads/writes, not pipelining thousands of commands, this is acceptable. The generous free tier was also a practical factor.

File: `config/redis.js` + `package.json` (@upstash/redis dependency)

↳ Follow-up: "How would you migrate to self-hosted Redis if load increased?" — Change the `redis.js` config to use `ioredis` with a Redis container in `docker-compose.yml`. The rest of the codebase uses the same `get/set/del` interface, so only the client initialization changes.

24 The `sendSalesReport` function fetches invoices, generates a Puppeteer PDF, then calls `sendEmailWithAttachment().catch()`. Why not `await` the email?

Answer: PDF generation with Puppeteer takes 2-3 seconds — the email itself adds another 1-2 seconds for the Mailjet API call. If both were awaited, the client would wait 3-5 seconds for the response, which is a poor experience. By calling without `await` and responding immediately with 200, the user sees "Report sent!" instantly. The PDF generation and email delivery continue in the background. The risk is that if the Node.js process crashes or restarts after the 200 response but before the email is sent, the email is silently lost. A production improvement would be a job queue (Bull/BullMQ with Redis) that persists the email job and retries on failure. For this project's scale, fire-and-forget is an acceptable tradeoff.

File: `invoice.controller.js` (lines 230-237)

↳ Follow-up: "How would you implement email delivery guarantee?" — Use Bull queue backed by Redis. When the report is requested, push a job to the queue with the PDF data. A separate worker process dequeues and sends. Bull provides automatic retries with exponential backoff on failure.

25 Looking at the entire project, if you had to redesign one thing differently, what would it be and why?

Answer: The stock management during invoice creation. Currently, I loop through items sequentially, read product stock, validate, then write — this is NOT atomic across multiple products. If the server crashes after deducting stock for Item 1 but before saving the invoice, the stock is permanently inconsistent. The right solution is MongoDB's multi-document transactions (available since v4.0). Wrap the entire createInvoice operation in a session: `const session = await mongoose.startSession(); session.startTransaction(); try { ...all writes... await session.commitTransaction(); } catch { await session.abortTransaction(); }` . If anything fails, ALL writes are rolled back atomically. I didn't implement this initially for simplicity, but it's the correct solution for a financial application where data integrity is critical.

📁 File: `invoice.controller.js` (lines 19-43)

↳ Follow-up: "What's the performance impact of MongoDB transactions?" — Transactions add latency (network round-trips for coordinator) and lock contention. For JusBill's expected load (single-user per shop), this is negligible. For high-volume multi-user scenarios, you'd want to profile carefully.

8.1 Complete API Reference

Method	Endpoint	Auth?	Controller Function	Key Behavior
POST	/api/auth/register	No	register()	Hash password, generate shopId (nanoid), sign JWT, set HttpOnly cookie
POST	/api/auth/login	No	login()	bcrypt.compare(), sign JWT, set cookie
POST	/api/auth/logout	No	logout()	clearCookie with maxAge:0
GET	/api/auth/me	 Yes	verifyUser()	Returns user without password. Cache-Control: no-store
GET	/api/products	 Yes	getProducts()	Redis cache (5min). Supports ?search= query
POST	/api/products	 Yes	createProduct()	Save product, invalidate products cache
PUT	/api/products/:id	 Yes	updateProduct()	findOneAndUpdate with shopId scope, invalidate cache
DELETE	/api/products/:id	 Yes	deleteProduct()	findOneAndDelete with shopId scope, invalidate cache
GET	/api/products/public/:shopId	No	getPublicProducts()	Filters isRawMaterial. Exposes only safe fields via .select()
GET	/api/products/top-selling	 Yes	getTopSellingProducts()	\$unwind items + \$group by productId. Redis cache (10min)

GET	/api/invoice	 Yes	getInvoices()	Promise.all for invoice list + aggregation stats. Redis cache
POST	/api/invoice	 Yes	createInvoice()	Sequential stock deduction, atomic counter, cache invalidation ×3
PUT	/api/invoice/:id/status	 Yes	updatePaymentStatus()	Cannot revert Paid→Unpaid. Enum validation.
POST	/api/invoice/send-email	 Yes	sendInvoiceEmail()	Base64 PDF buffer → Mailjet. Fire-and-forget.
POST	/api/invoice/send-report	 Yes	sendSalesReport()	Puppeteer PDF → Mailjet. Fire-and-forget.
GET/POST/PUT/DELETE	/api/customers	 Yes	CRUD	GSTIN regex validation. Unique {shopId,mobile} and {shopId,email}
POST	/api/purchases	 Yes	createPurchase()	Atomic counter, save purchase, \$inc stock for each item
DELETE	/api/purchases/:id	 Yes	deletePurchase()	Revert stock (\$inc: -qty), decrement counter (known issue)
POST	/api/chat	 Yes	chat()	Build context from DB, call Groq API (Llama 3.1-8b), return response
PUT	/api/users/profile	 Yes	updateProfile()	Multer+Cloudinary upload, save URLs, invalidate user cache
GET	/health	No	inline	Returns {status:"ok", timestamp}. Used by Docker healthcheck.

8.2 Things to Revise Before Interviews

🔒 AUTHENTICATION

- JWT payload: `{id, shopId, iat, exp}`
- Cookie: httpOnly, sameSite:lax, 15 days
- bcrypt cost factor = 10 (~100ms)
- nanoid(10) for shopId generation
- Error 11000 = MongoDB duplicate key
- Cache-Control: no-store on /me endpoint

⚡ CACHING

- Pattern: Cache-Aside (Lazy Loading)
- Key prefix: "jusbill_v3:" (version-based)
- Products TTL: 300s (5 min)
- Analytics TTL: 600s (10 min)
- Purchases TTL: 3600s (1 hr)
- Redis failure → fallback to MongoDB
- Non-blocking invalidation: `.catch(()⇒{})`

⏸ KEY DESIGN DECISIONS

- Multi-tenancy via shopId on every query
- Customer data denormalized in Invoice
- Counter collection for sequential IDs
- isRawMaterial hidden from public API
- Stock deduction: sequential loop, not Promise.all
- Paid→Unpaid revert blocked at controller

⚠ KNOWN ISSUES TO ADMIT

- CORS allowedOrigins = ["*"] — too permissive
- No MongoDB transactions in createInvoice
- updateProfile has mass assignment risk
- Counter decrement on delete → ID reuse
- Customer search uses \$regex not \$text index
- Puppeteer: new browser per PDF request
- secure: false cookie (HTTP testing artifact)

📁 KEY FILES TO KNOW

- **auth.controller.js** — register, login, JWT
- **middleware/auth.js** — JWT verification
- **invoice.controller.js** — most complex
- **chat.controller.js** — AI integration
- **utils/cacheHelper.js** — cache pattern
- **middleware/cache.js** — invalidation
- **config/redis.js** — key namespacing
- **utils/pdfGenerator.js** — Puppeteer
- **config/upload.js** — Multer+Cloudinary
- **context/AuthContext.jsx** — frontend auth

💡 ONE-LINERS TO REMEMBER

- "HttpOnly cookies prevent XSS token theft"
- "Promise.all for parallel reads, sequential loop for dependent writes"
- "Denormalized customer data = historical accuracy"
- "Version prefix = zero-downtime cache busting"
- "\$ne: true not false for backward compatibility"
- "Redis failure is non-fatal — fallback to DB"
- "Groq on backend = API key never exposed to browser"
- "Puppeteer --no-sandbox = Docker compatibility"

8.3 Common Mistakes While Explaining This Project

❌ **Don't say:** "I used JWT for authentication" — Too generic.

✅ **Say:** "I stored JWT in an HttpOnly cookie to prevent XSS. The payload contains only id and shopId for multi-tenancy. It's verified on every protected route by my auth.js middleware."

❌ **Don't say:** "I used Redis for caching" — Too generic.

✅ **Say:** "I implemented a Cache-Aside pattern using Upstash Redis with version-prefixed keys (jusbill_v3:). Cache misses fall through to MongoDB. Invalidation happens non-blockingly on mutations to avoid delaying API responses."

❌ **Don't say:** "PDF generation uses Puppeteer" — Too generic.

✅ **Say:** "Server-side PDF with Puppeteer uses headless Chrome with --no-sandbox for Docker compatibility. Each PDF is generated on-demand, the buffer returned, and emails are sent fire-and-forget in the background."

✅ **How to handle questions about known bugs:** Acknowledge them confidently — "I'm aware of this — the correct fix is [specific solution]. I made this tradeoff because [reason]." Interviewers respect engineers who can critically evaluate their own code.

✅ **How to frame the project:** "JusBill is a multi-tenant SaaS platform serving Indian retailers. I designed it around the constraint that every database query must be tenant-scoped by shopId — which is embedded in the JWT and available as req.user.shopId throughout the request lifecycle."

8.4 Architecture Summary (Memorize This)

JusBill in 5 sentences

JusBill is a full-stack multi-tenant SaaS platform built with React + Vite on the frontend and Node.js + Express on the backend. Users are authenticated via JWT stored in HttpOnly cookies, with each user isolated by their unique shopId used to scope all MongoDB queries. The application uses a Cache-Aside pattern with Upstash Redis (5-minute TTL for products and invoices) to reduce DB load, with version-prefixed keys for zero-downtime cache busting. Key features include server-side PDF generation via Puppeteer, image upload to Cloudinary CDN via Multer, transactional email via Mailjet's HTTP API, and AI-powered business insights using Groq's Llama 3.1-8b model with real-time shop data as context. The entire application is containerized with Docker Compose (dev/prod profiles) and served through Nginx reverse proxy on a Google Cloud VM with SSL termination.